



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



“INSTALACIÓN DE UN SGBD”

EQUIPO:

DANAET MONTSERRATH GONZÁLEZ MARTÍNEZ

GARCIA CASTILLO MARIO

ISLAS RAMIREZ DIEGO

GRUPO: 3CV2

MATERIA: BASE DE DATOS

PROFESOR: GABRIEL HURTADO AVILÉS

Ejercicio 1. Control de versiones con Git y GitHub

El **control de versiones** es un software que registra cada cambio realizado en el código a medida que se edita y los guarda para su recuperación, de manera segura, eficiente y ágil a lo largo del tiempo.

A medida que un desarrollador edita código, el sistema de control de versiones toma una instantánea de los archivos. A continuación, guarda esa instantánea de forma permanente para que se pueda recuperar más adelante si es necesario.

Debido a que las demandas de los usuarios aumentan, el control de versiones ayuda a los equipos a trabajar juntos y entregar a tiempo. Concretamente, resuelve los problemas de colaboración y organización mediante los siguientes puntos:

- Ayuda a distribuir soluciones a tiempo.
- Expone el aprendizaje de nuevas herramientas a los/as siguientes compañeros/as.
- Evita la reproducción de errores para quienes retomen el código.
- Unifica el flujo de trabajo en el proceso de desarrollo.
- Sincronizar las versiones permite codificar sin que los cambios se solapen.
- Las modificaciones se guardan en un historial, un registro útil para revisar todas las etapas del proyecto, corregir y recuperar código con facilidad.
- El control de versiones automatiza tareas, por lo que agiliza el flujo de trabajo.

Por otra parte, tenemos que *“Git: una herramienta local y de código abierto”*. Es un software que se instala en la computadora y su función principal es la de gestionar versiones del código de forma local y distribuida. Es completamente independiente de cualquier plataforma externa, lo que implica que se puede utilizar sin necesidad de contar con servicios de terceros ni conexión a internet.

Mientras que, *“GitHub: una plataforma en la nube para alojar repositorios Git”*. Es un servicio en línea y su objetivo es alojar repositorios Git en un repositorio remoto accesible desde cualquier lugar. Además, añade capas de colaboración, automatización y gestión de proyectos, convirtiéndose en un entorno de trabajo colaborativo para equipos de desarrollo.

En este contexto, una **confirmación (commit)** funciona como tomar una fotografía de los archivos o guardar el estado del código en algún momento con una nota explicativa para poder revisarlo o recuperar ese avance más adelante, como por ejemplo cuando se crea el archivo **Dockerfile** para definir cómo se ejecutará la aplicación y se hace un **commit** con el mensaje **“Agregar Dockerfile para la configuración del entorno del proyecto”**.

Por su parte, una **rama (branch)** es una línea independiente de trabajo que permite modificar o agregar características al código sin alterar la rama principal ni interferir con otros trabajos; por ejemplo, si se necesita agregar autenticación de usuarios mediante **Docker** sin romper la versión funcional de la aplicación, se crea la rama **auth-docker** para desarrollar y probar la funcionalidad de forma segura.

Posteriormente, la **fusión (merge)** integra el trabajo realizado por separado de vuelta a la línea (rama) principal, lo que se aborda ejecutando la integración de la rama **auth-docker** hacia **main** una vez que los contenedores de autenticación funcionan correctamente y pasaron todas las pruebas.

Sin embargo, **un conflicto de fusión (merge conflict)** ocurre cuando dos personas modifican la misma línea de un mismo archivo de manera diferente en distintas ramas, haciendo que se detenga la fusión y pidiendo al equipo que decida qué versión dejar; por ejemplo, si en la rama principal se modificó la línea 5 del **Dockerfile** para usar una versión más reciente, y al mismo tiempo una persona modifica esa misma línea a una versión atrasada en su rama. Al intentar unir las dos ramas, Git genera un conflicto para que se decida qué versión se conserva antes de completar la integración.

Para gestionar y revisar estos cambios antes de unirlos, un **Pull Request** permite proponer cambios en un repositorio y solicitar que se fusionen con una rama específica, lo cual se aborda abriendo una solicitud desde la rama **auth-docker** hacia **main** en GitHub, adjuntando la descripción del entorno configurado para que los líderes del proyecto revisen el código e inicien las pruebas automatizadas.

Adicionalmente, el **archivo .gitignore** es una lista de instrucciones en Git donde le indicas qué archivos o carpetas debe ignorar por completo para evitar subir datos innecesarios, pesados o confidenciales; esto se aborda agregando líneas como **node_modules**, **.env** y los volúmenes locales de **Docker** al **archivo .gitignore**, garantizando que las credenciales de la base de datos y los paquetes compilados no se suban por accidente al repositorio remoto.

Finalmente, el **archivo README** explica de qué trata la aplicación, cómo se instala y cómo se usa a forma de manual, lo cual se aborda incluyendo en el archivo **README.md** una sección clara de instalación que indica al equipo: **“Para levantar el proyecto con Docker, se clona el repositorio y se ejecuta el comando docker-compose up –build en la terminal”**.

Todo esto se enmarca en un **flujo de trabajo** basado en ramas, el cual consiste en mantener la rama principal (normalmente **main** o **master**) siempre limpia, estable y lista para desplegarse a producción, mientras que todo trabajo nuevo se realiza en ramas secundarias o de tema.

Este proceso sigue una secuencia estructurada:

- **Creación de la rama:** Antes de escribir el código, se crea una rama corta y descriptiva a partir de la rama principal.

- **Desarrollo aislado:** El desarrollador realiza cambios, añade avances y realiza commits exclusivamente dentro de su rama sin alterar el trabajo de los demás ni poner en riesgo la versión estable.
- **Solicitud de integración:** Cuando la solución o característica esta lista, se abre una solicitud de extracción o Pull Request para proponer la incorporación de esos cambios a la rama principal.
- **Fusión y despliegue:** Una vez aprobada la propuesta, la rama se fusiona (merge) con la rama principal y se elimina la rama secundaria para mantener el repositorio ordenado.

Por último, la **revisión entre pares (peer review)** mediante herramientas como los Pull Requests de GitHub es una etapa crucial antes de la fusión por las siguientes razones clave:

- **Garantía de calidad y prevención de errores:** Permite que otros desarrolladores examinen el código en busca de errores, cuellos de botella de rendimiento o fallos de seguridad antes de que lleguen a la rama principal y afecten a los usuarios finales.
- **Mantenimiento de estándares del proyecto:** Asegura que el nuevo código cumpla con las guías de estilo, la arquitectura del software y las mejores prácticas acordadas por el equipo.
- **Intercambio de conocimiento:** Expone el aprendizaje a todo el equipo. Los compañeros entienden que cambios se están introduciendo en el sistema, mientras que el autor del código puede recibir sugerencias alternativas para mejorar sus habilidades.
- **Fomento del debate y la colaboración:** Abre un espacio para discutir soluciones alternativas, proponer mejoras adicionales o solicitar pruebas antes de que el código sea considerado definitivo.

FUENTES BIBLIOGRAFICAS:

- BillWagner. (2025, 27 de octubre). *¿Qué es el control de versiones?* Microsoft Learn. <https://learn.microsoft.com/es-es/devops/develop/git/what-is-version-control>
- Chacon, S., & Straub, B. (2014). *Pro Git* (2.^a ed.). Apress. <https://git-scm.com/book/es/v2/Ramificaciones-en-Git-Flujos-de-Trabajo-Ramificados>
- Delgado, J. A. (2026). *Git vs GitHub: diferencia, usos y por qué son clave*. DKS Digital. <https://dks.digital/mx/blog/programacion/git-vs-github-diferencia-usos-y-por-que-son-clave/>
- GitHub. (2026). *Acerca de Git*. Documentación de GitHub. <https://docs.github.com/es/get-started/using-git/about-git>
- GitHub. (2026). *Flujo de trabajo de GitHub (GitHub flow)*. Documentación de GitHub. <https://docs.github.com/es/get-started/using-github/github-flow>
- Sanugommula, H. (2022). Comprehensive study of Git and Github: Implementing themes learning objectives in modern education. *Journal of Media & Management*, 4(6), 1–6. https://www.researchgate.net/publication/387160358_Comprehensive_Study_of_Git_and_Github_Implementing_Themas_Learning_Objectives_in_Modern_Education
- Tokio School. (2026). *Control de versiones: qué es, tipos y herramientas*. <https://www.tokioschool.com/noticias/control-versiones/>
- We Learn Data. (2023). *Git y GitHub: Conceptos principales*. <https://welearndata.com/git/git-github-conceptos-principales/>